

Lecture 6

Regression & Supervised Machine Learning for CLV

Linear Regression, Regularization, Trees, and Business Analytics Intuition

Armando Vieira

Tartu, Spring 2026

- Introduction to supervised learning and prediction problems in business analytics.
- Underfitting, overfitting, and the bias--variance trade-off.
- Regularization: why it matters and how Ridge and LASSO help control complexity.

Main question

How do we build models that are useful for decisions **without fitting noise**?

Practical lens

For business data, the winner is often not the fanciest model. It is the model that **generalizes, explains enough, and supports action**.

Supervised vs. unsupervised learning

Supervised learning

- Learn a mapping from inputs X to target y
- Requires labeled examples
- Main tasks:
 - Regression: predict a number
 - Classification: predict a class

Unsupervised learning

- No target variable
- Goal is to find structure or groups
- Typical tasks:
 - Clustering
 - Dimensionality reduction
 - Anomaly detection

Business examples

Customer segmentation is usually unsupervised. Predicting churn, revenue, demand, or CLV is supervised.

Regression vs. classification

Regression

Output is continuous.

- Future customer value
- Next-month demand
- Delivery time
- Sales uplift

Classification

Output is categorical.

- Will the customer churn?
- Fraud or not fraud?
- High-value or low-value segment?
- Will a customer respond to a campaign?

Rule of thumb

If the answer is a **number**, think regression. If the answer is a **label**, think classification.

Linear regression: the basic idea

We approximate the relationship between features and an outcome with:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_k x_k + \varepsilon$$

- β_0 : intercept or baseline prediction
- β_j : expected change in y when feature x_j increases by 1, **holding the others fixed**
- ε : residual error or noise

Training objective

Choose coefficients that minimize prediction error, usually the **sum of squared errors**.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Why linear models still matter in business analytics

- Fast to fit and easy to explain to managers.
- Strong baseline for tabular data.
- Useful when you care about **direction** and **approximate magnitude** of effects.
- Easy to connect to business questions such as:
 - Which features are associated with CLV?
 - Is discounting helping or hurting margin?
 - Does website activity predict future spend?

But...

Business datasets often have **many correlated features**: email clicks, visits, ad spend, campaign touches, discounts, and support interactions. That is exactly where **regularization** becomes useful.

Train, validation, and test

Split	Purpose
Train	Fit model parameters from data
Validation	Tune hyperparameters and compare model choices
Test	Final unbiased estimate of real-world performance

Typical split: **70% train, 15% validation, 15% test**

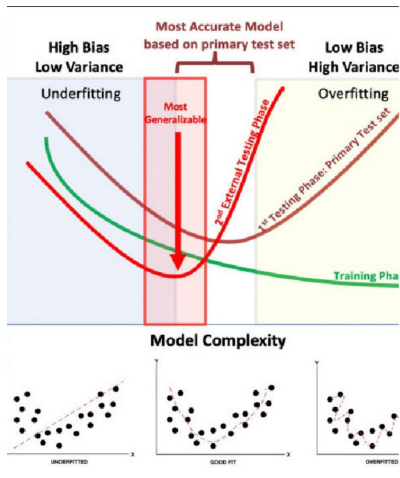
Why this matters

A model can look excellent on training data because it memorized noise. The test set asks the only question that matters: **does it work on unseen data?**

Tip

When the sample is not large, use **k -fold cross-validation** to estimate performance more reliably.

Bias-variance trade-off



High bias

Model is too simple. It misses real patterns. This is **underfitting**.

High variance

Model is too flexible. It learns noise. This is **overfitting**.

Goal

Find the middle region where the model is complex enough to learn signal but not so flexible that it memorizes random fluctuations.

Regularization: what problem are we solving?

- Without regularization, a model may chase tiny patterns in the training sample that will never repeat.
- This is especially common when:
 - there are many features,
 - predictors are highly correlated,
 - the sample is not large,
 - we create many interactions or polynomial terms.

Main idea

Regularization adds a **penalty for complexity**. The model is still trying to fit the data, but it is also rewarded for staying simple.

Business intuition

A model that says campaign A matters a lot in one month and almost not at all in the next month is often unstable. Regularization pushes the model toward **more stable, more generalizable estimates**.

OLS vs. Ridge vs. LASSO

Method	Best when	What it does
OLS	Few strong predictors, low collinearity	No penalty; focuses only on fit to the sample
Ridge	Many weak or correlated predictors	Shrinks coefficients toward zero but usually keeps all variables
LASSO	Only a subset of predictors probably matters	Shrinks coefficients and can set some exactly to zero

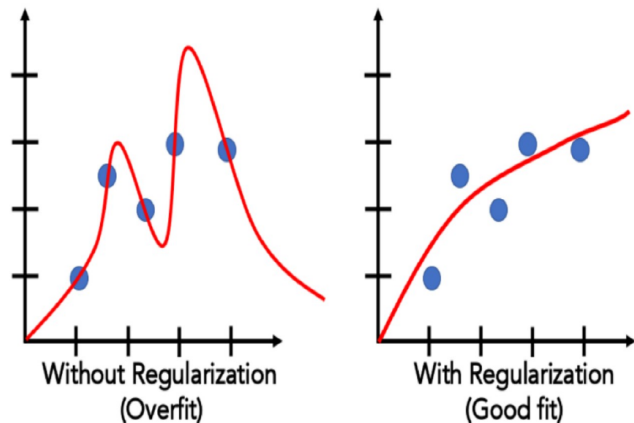
Penalty terms

Ridge: $SSE + \lambda \sum_j \beta_j^2$ LASSO: $SSE + \lambda \sum_j |\beta_j|$

Good default intuition

Use **Ridge** when many variables share information. Use **LASSO** when you want a smaller, easier-to-explain feature set

Visual intuition: overfit vs. regularized fit



Left panel

A very wiggly curve fits the sample closely, but it is reacting to noise.

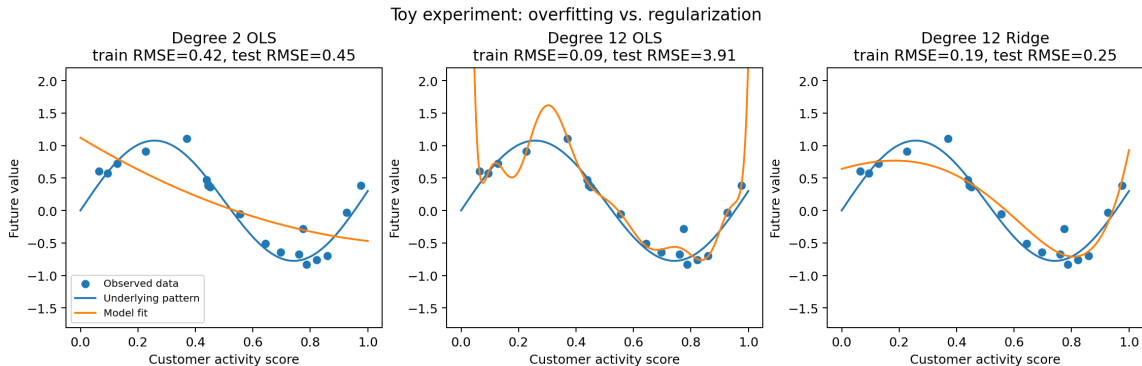
Right panel

A smoother curve ignores small fluctuations and captures the broader pattern.

Takeaway

Regularization does **not** mean “make the model bad.” It means **stop spending complexity on noise**.

Toy experiment 1: when a flexible model overfits

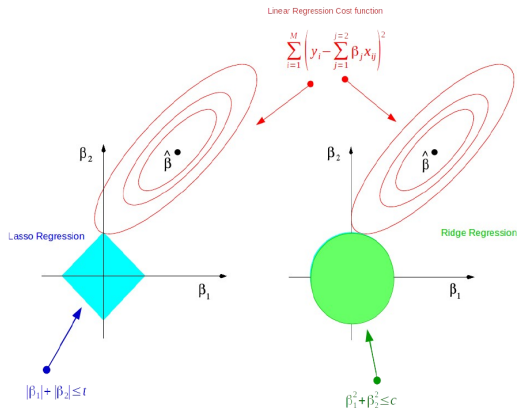


How to read this

Degree-12 OLS fits the observed sample extremely well, but its test error explodes. Adding Ridge keeps the flexible model, but limits how wildly it can bend.

Why Ridge and LASSO behave differently

Dimension Reduction of Feature Space with LASSO



Ridge (L2)

The constraint region is round. The optimum usually lands away from the axes, so coefficients become **smaller** but not exactly zero.

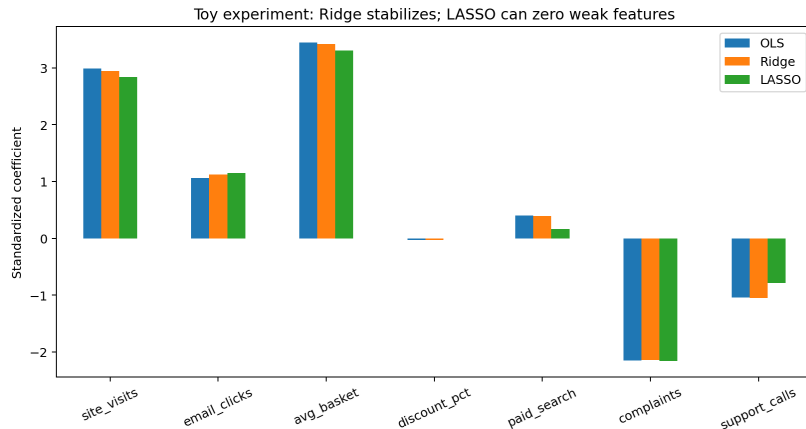
LASSO (L1)

The constraint region has corners. The optimum often touches a corner, which makes some coefficients **exactly zero**.

Interpretation

Ridge is usually better for **stability**.

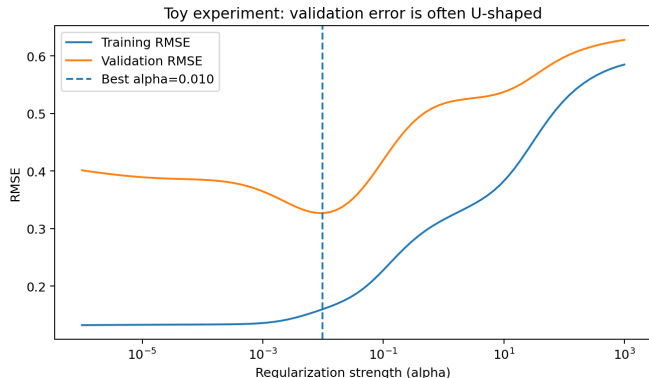
Toy experiment 2: a business-style coefficient comparison



Interpretation

On correlated marketing and service features, Ridge spreads weight more smoothly across related variables. LASSO keeps the strongest signals and drops some weak ones to zero.

How do we choose the penalty strength α or λ ?



- Very small α : almost no penalty, so the model can overfit.
- Very large α : coefficients are forced too close to zero, so the model underfits.
- The best value is selected using **validation data** or **cross-validation**.

Code pattern: regularization done properly

```
import numpy as np
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import RidgeCV, LassoCV

alphas = np.logspace(-3, 2, 30)
ridge = make_pipeline(StandardScaler(), RidgeCV(alphas=alphas, cv=5))
lasso = make_pipeline(StandardScaler(),
                      LassoCV(alphas=alphas, cv=5, max_iter=5000))

ridge.fit(X_train, y_train)
lasso.fit(X_train, y_train)
print(ridge[-1].alpha_, lasso[-1].alpha_)
```

Important habits

Scale first. Tune with CV. Compare against ordinary linear regression.

Business analytics takeaways for regularization

- **Prediction goal:** Ridge is often the safest starting point when many variables are correlated.
- **Interpretation goal:** LASSO can be attractive when you need a shorter feature list.
- **Attribution warning:** do not over-interpret individual coefficients when variables overlap heavily.
- **Scaling matters:** a variable measured in euros and another measured in clicks should not face different penalties just because of units.
- **More features is not always better:** additional variables can increase variance more than they add signal.

Useful sentence in reports

“We selected the regularization strength using cross-validation to improve out-of-sample performance and reduce coefficient instability.”

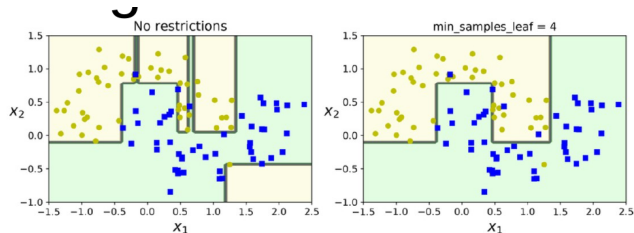
Decision trees: a different kind of flexibility

- Trees split the feature space into regions using rules like `recency < 30 days` or `avg_basket > 70`.
- They automatically capture nonlinear patterns and interactions.
- They are excellent for tabular business data, but a fully grown tree can overfit badly.

Tree regularization is different

We do not shrink coefficients. Instead, we limit tree complexity with hyperparameters such as: `max_depth`, `min_samples_split`, and `min_samples_leaf`.

regularization figure



No restrictions

The tree keeps carving the space into tiny boxes and starts matching noise.

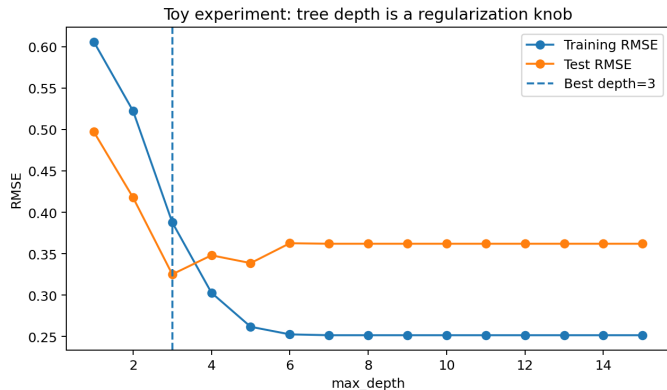
With restrictions

Larger leaves force the model to average more observations, which usually improves generalization.

Practical message

Tree regularization is still about the same idea: **allow enough flexibility to learn signal, but not enough to memorize noise.**

Toy experiment 3: tree depth is also a regularization knob



- As depth increases, training error usually keeps falling.
- Test error often improves only up to a point, then stops improving or gets worse.
- That best depth is the tree analogue of a good regularization level.

Code pattern: regularized tree and random forest

```
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor

tr = DecisionTreeRegressor(max_depth=4,
                           min_samples_leaf=20,
                           random_state=42)
rf = RandomForestRegressor(n_estimators=300,
                           max_depth=8,
                           min_samples_leaf=10,
                           random_state=42,
                           n_jobs=-1)

tr.fit(X_train, y_train)
rf.fit(X_train, y_train)
```

Tip

Trees usually do not need scaling. Forests reduce variance by averaging many regularized trees.

CLV: where these models become useful

Historical CLV

Summarizes what the customer already generated.

$CLV = \text{Average Purchase Value} \times \text{Purchase Frequency}$

Predictive CLV

Estimates future value using behavior such as:

- Recency
- Frequency
- Monetary value
- Returns rate
- Channel, campaign, and churn signals

Model choice intuition

Linear models are useful when you want transparency. Trees and forests are useful when patterns are nonlinear and interaction-heavy.

Code example: a practical CLV workflow

```
features = [  
    "recency_days", "orders_6m", "avg_order_value",  
    "discount_share", "returns_rate", "channel_paid"  
]  
  
X = customers[features]  
y = customers["clv_next_12m"]  
  
from sklearn.model_selection import train_test_split  
from sklearn.metrics import mean_absolute_error  
from sklearn.ensemble import RandomForestRegressor  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)  
  
model = RandomForestRegressor(n_estimators=400, random_state=42)  
model.fit(X_train, y_train)  
pred = model.predict(X_test)  
print("MAE:", mean_absolute_error(y_test, pred))
```

Tips, tricks, and useful insights

- Start with a **simple baseline**. If linear regression is already strong, extra complexity may not pay off.
- Always state the **prediction horizon**: CLV for the next 6 months is a different target from CLV for the next 12 months.
- Use **MAE** when you want an error in business units such as euros or dollars.
- A big gap between train and test performance usually means **overfitting**.
- For linear models with penalties, **scale first**. For trees, scaling is usually unnecessary.
- Beware of **data leakage**: never use future information as an input feature.
- Feature engineering often matters more than moving from one advanced model to another.

One-slide summary

- 1 Supervised learning predicts labeled outcomes; regression predicts numbers and classification predicts classes.
- 2 Linear regression remains a strong baseline for business analytics, especially when explanation matters.
- 3 Regularization helps when models are too flexible, when predictors are correlated, or when the feature space is large.
- 4 Ridge usually improves stability; LASSO can also do feature selection.
- 5 Decision trees need their own regularization through depth and leaf-size controls.
- 6 For CLV and similar tasks, choose the model that **generalizes well and supports better business decisions**.